INGENIERÍA EN INFORMÁTICA · 3° AÑO

AUTÓMATAS FINITOS

Clase : AFD, AFN y Construcción de Thompson

Agenda de la Clase

¿Qué vamos a ver hoy?

01 — 10 min

¿Qué es un Autómata?

Definición, tipos, aplicaciones

02 — 20 min

Autómatas Finitos Deterministas

AFD, definición formal, ejemplos

03 — 15 min

Tablas de transición

Notación tabular, implementación

04 — 20 min

Autómatas Finitos No Deterministas

AFN, diferencias con AFD

05 — 20 min

Construcción de Thompson

ER → AFN, algoritmo paso a paso

06 — 5 min

Síntesis y cierre

Resumen y próxima clase

01

SECCIÓN

¿Qué es un Autómata?

Modelos abstractos de computación

¿Qué es un Autómata?

Modelo matemático abstracto de una máquina

Un autómata es un modelo matemático abstracto de una máquina que realiza operaciones de manera automática. Se utiliza para representar y comprender el comportamiento de sistemas computacionales y para resolver problemas en múltiples campos.

Estructura básica de un autómata

Todo autómata tiene:

- Estados (situaciones posibles del sistema)
- Entradas (símbolos del alfabeto Σ)
- Transiciones (reglas de cambio de estado)
- Estado inicial y estados de aceptación

Analogía: el interruptor de luz

Un interruptor tiene dos estados: ON y OFF.
Cuando el usuario pulsa el botón, el efecto depende del estado actual:

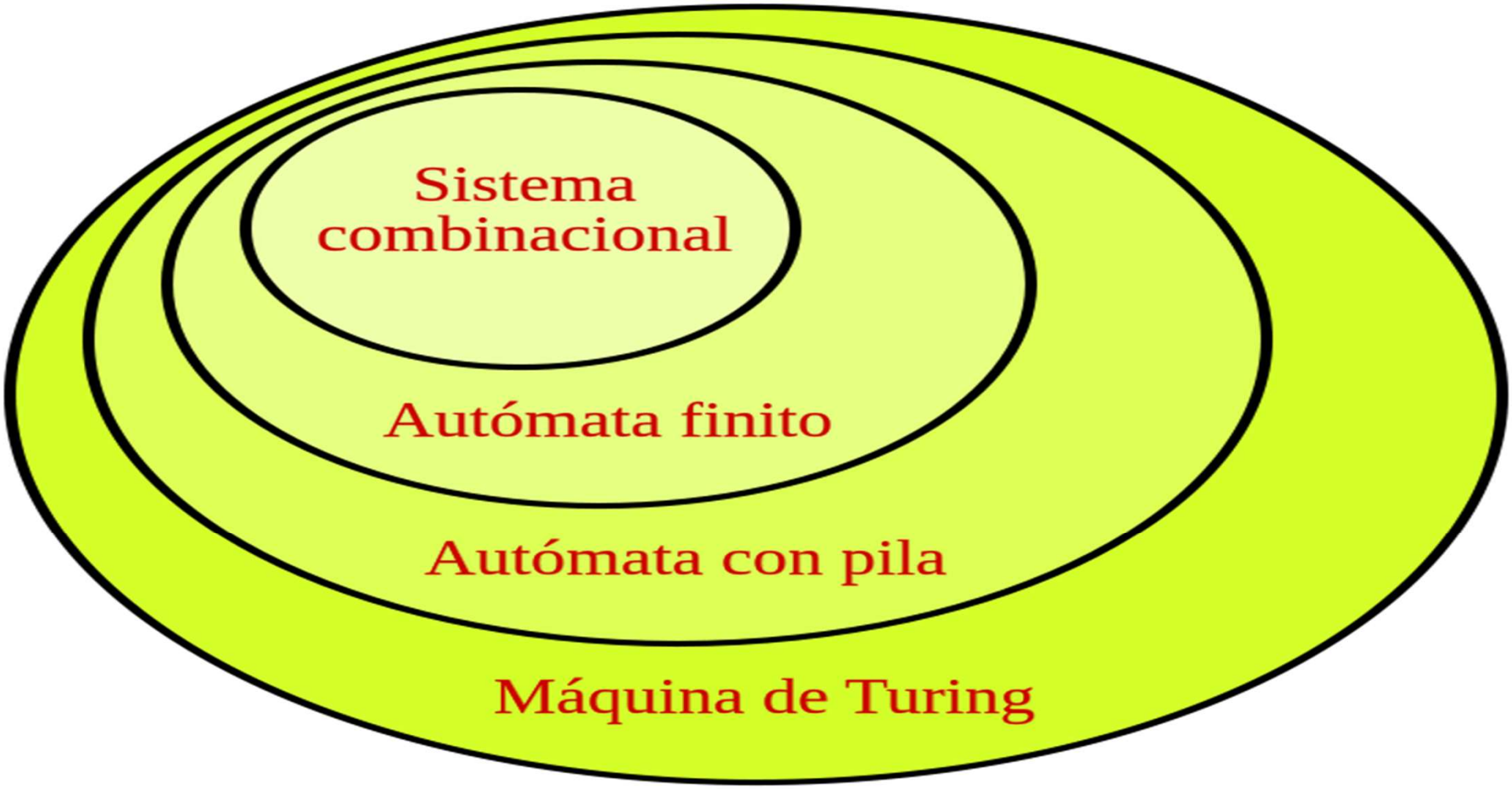
ON → pulsar → OFF
OFF → pulsar → ON

Este es el autómata más simple posible.

Tipos de autómatas (de menor a mayor potencia computacional):

Sistema combinacional	Autómata Finito (AF)	Autómata de Pila (AP)	Máquina de Turing (MT)
Sus salidas son función exclusiva del valor de sus entradas	Reconoce lenguajes regulares (que puede ser definido por una expresión regular)	Reconoce lenguajes libres de contexto	Reconoce lenguajes recursivamente enumerables

TIPOS DE AUTOMATAS



Aplicaciones de los Autómatas Finitos

¿Dónde se usan en la práctica?

Analizador Léxico (Compiladores)

El primer paso de un compilador es el análisis léxico: divide el código fuente en tokens (palabras clave, identificadores, números, operadores). Cada token es reconocido por un AF.

Protocolos de Comunicación

Los protocolos de red (TCP, HTTP) se modelan como autómatas: cada estado representa una fase del protocolo y las transiciones ocurren al recibir mensajes.

Diseño de Circuitos Digitales

Los sistemas combinacionales y secuenciales se modelan como autómatas. Los sistemas combinacionales (álgebra de Boole) son un caso especial sin memoria de estado.

Sistemas con Estados Finitos

Semáforos, cajeros automáticos, controladores de ascensores, máquinas expendedoras. Cualquier sistema con comportamiento predecible basado en estados.

Búsqueda y Reconocimiento de Patrones

El motor de expresiones regulares de grep, Python re, y editores de texto implementan un AF internamente para encontrar patrones en texto.

Lingüística Computacional e IA

Reconocimiento de lenguaje natural, morfología computacional, y sistemas de diálogo usan autómatas para modelar estructuras lingüísticas.

02

SECCIÓN

Autómatas Finitos Deterministas (AFD)

Definición formal y ejemplos

Definición Formal de un AFD

La quintupla que define completamente un autómata

Un AFD M se define como una quintupla: $M = (Q, \Sigma, \delta, q_0, F)$

Q

Conjunto de estados

Conjunto finito de todos los estados posibles del autómata.
Ej: $Q = \{ q_0, q_1, q_2, q_3 \}$

Σ

Alfabeto de entrada

Conjunto finito de símbolos que el autómata puede leer.
Ej: $\Sigma = \{ 0, 1 \}$ o $\Sigma = \{$ letra, dígito $\}$

δ

Función de transición

$\delta: Q \times \Sigma \rightarrow Q$
Dada un estado y un símbolo, devuelve el estado siguiente.
Ej: $\delta(q_0, a) = q_1$

q_0

Estado inicial

El estado donde comienza el procesamiento. $q_0 \in Q$.

F

Estados de aceptación

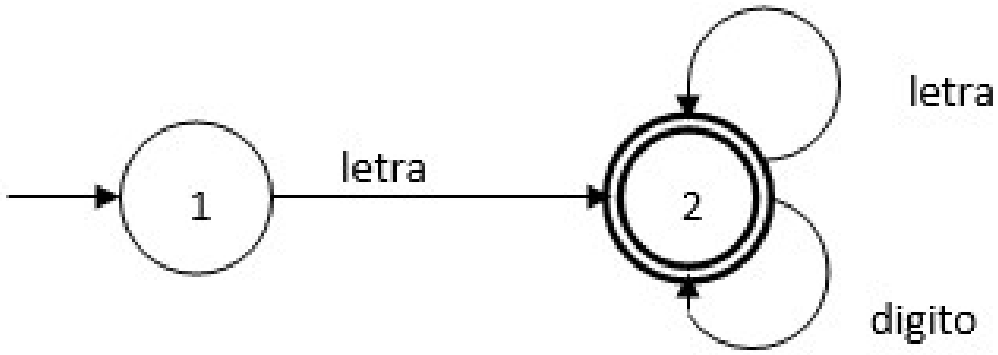
$F \subseteq Q$. Si al terminar de leer la cadena estamos en $F \rightarrow$ ACEPTA.

Ejemplo AFD: Reconocimiento de Identificadores

*identificador = letra (letra | dígito)**

```
Definición regular:  identificador = letra ( letra | dígito )*
```

Diagrama de transición:



Estado q_0 — Estado inicial

El autómata comienza aquí. Si el primer símbolo es una letra → va a q_1 (posible aceptación). Si NO es letra → va a qerr (error).

Estado q_1 — Estado de aceptación

Si llega una letra o dígito → permanece en q_1. Al encontrar '$' (fin) → ACEPTA el identificador.

Estado qerr — Estado de error

Trampa: una vez aquí, ya no es posible aceptar. Cualquier entrada mantiene en qerr.

PREGUNTA PARA LA CLASE

¿Cuál de estas cadenas SÍ sería aceptada por el AFD de identificadores?

A) '123abc' — empieza con dígito

C) '_nombre' — empieza con guión bajo

B) 'miVar2' — empieza con letra

D) '2do' — empieza con dígito

Ejemplo AFD: Transferencia de Dinero Electrónico

Modelando un protocolo con estados

El protocolo tiene tres participantes con sus propios autómatas que luego se combinan en el autómata producto:

Autómata del CLIENTE

Estado 0: Listo para pagar

→ pagar: manda dinero a tienda (Estado 1)

→ cancelar: manda dinero al banco (Estado 2)

Estado 1 y 2: terminales

Autómata de la TIENDA

Estado 0: Esperando pago

→ recibir pago: Estado 1 (tiene dinero)

→ suministrar bienes: Estado 2

→ liberar dinero al banco: Estado 3 (final)

Nota: ignora 'cancelar' (bucle sobre sí misma)

Autómata del BANCO

Estado 0: En espera

→ recibir cancelación: acredita al cliente (E1)

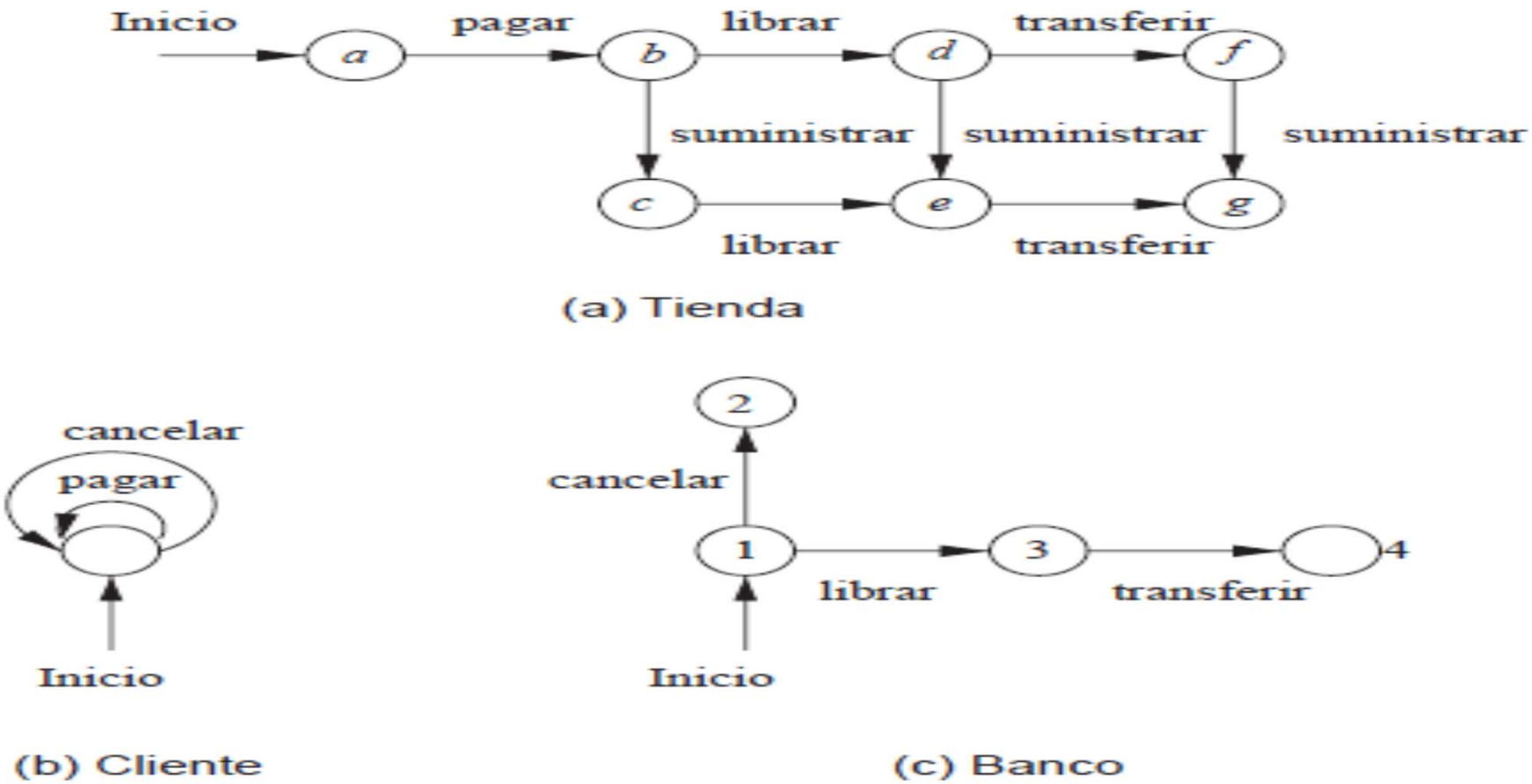
→ recibir liberación de tienda: acredita tienda (E2)

→ transferir: crea dinero electrónico (E3)

Estado 3: Estado de aceptación

Autómata producto: combina los 3 autómatas ejecutándolos 'en paralelo'. Los estados son tuplas (cliente, tienda, banco). Con 4 estados (tienda) × 7 estados (banco) = 28 estados totales.

Ejemplo AFD: Transferencia de Dinero Electrónico



Transiciones Faltantes: Ignorar Acciones

¿Qué pasa cuando un autómata recibe una entrada no prevista?

El problema

En el ejemplo de transferencia, la tienda no se ve afectada por la acción 'cancelar'. Si no definimos esa transición, cuando el cliente cancela el autómata de la tienda 'detiene su ejecución' y ya no puede procesar nada más.

Solución: transición a sí mismo (self-loop)

Agregamos un arco desde cada estado hacia sí mismo, etiquetado con 'cancelar'.
Así cuando ocurre 'cancelar', la tienda realiza una transición que no cambia su estado.
El autómata sigue activo y puede recibir más entradas.

Estado de error (trampa)

La alternativa es agregar un estado de error explícito (estado trampa).
Cualquier entrada no definida lleva al estado de error, desde el cual no se puede salir.
Esta es la implementación estándar en compiladores.

Regla: Un AFD debe tener una transición definida para CADA símbolo del alfabeto desde CADA estado.

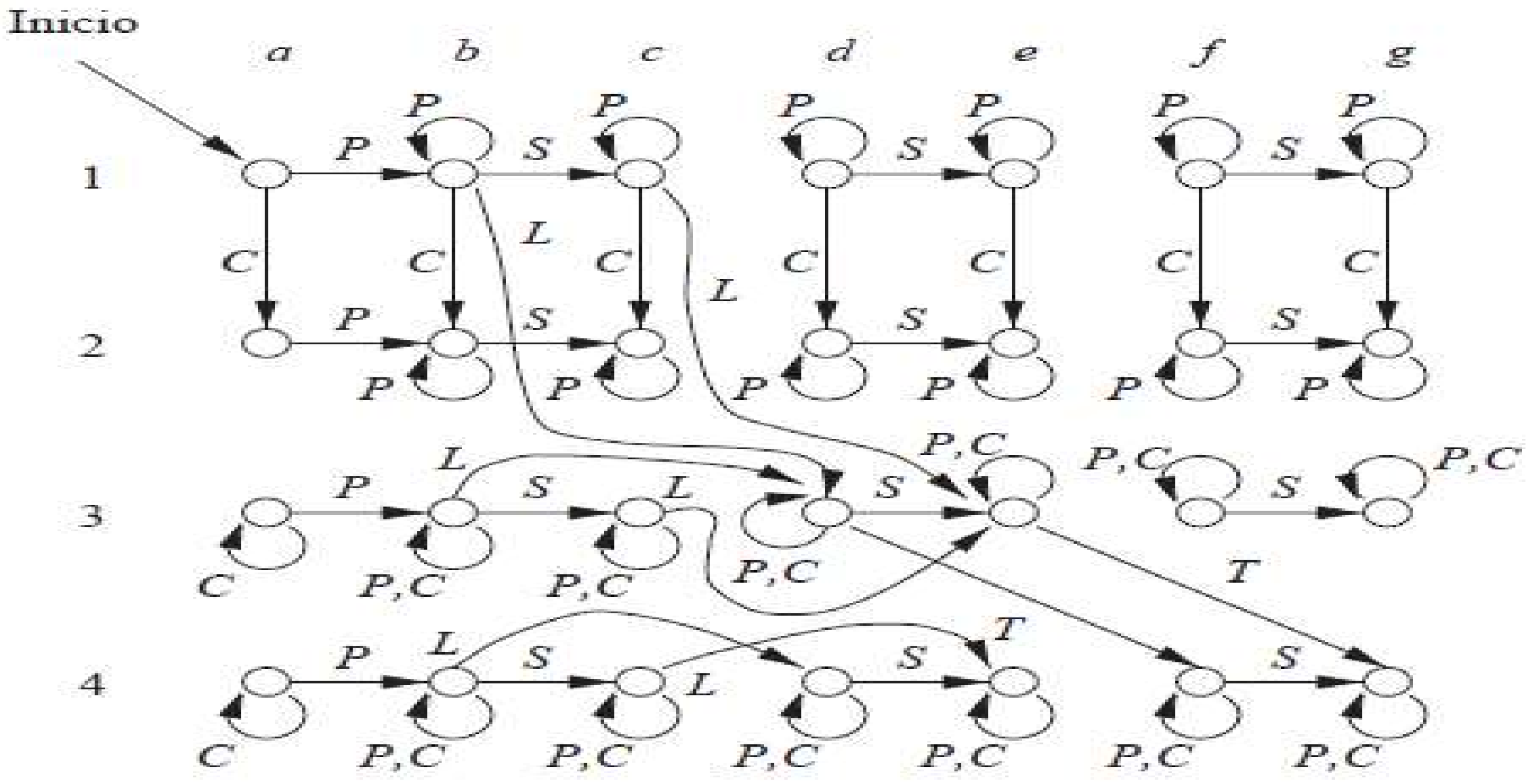
Sin self-loop (incompleto):

```
δ(E0, pago) = E1
δ(E1, bienes) = E2
// 'cancelar' no definido → ERROR
```

Con self-loop (completo):

```
δ(E0, pago) = E1
δ(E0, cancelar) = E0   // self-loop
δ(E1, cancelar) = E1   // self-loop
```

Ejemplo AFD: Transferencia de Dinero Electrónico



03

SECCIÓN

Tablas de Transición

Representación tabular del AFD

Tabla de Transiciones

Representación tabular del AFD para números exponenciales

La tabla de transiciones es una representación alternativa al diagrama. Las filas son estados, las columnas son símbolos del alfabeto, y cada celda contiene el estado destino δ(estado, símbolo).

```
ER del número:   signo? dígitos ( .dígitos )? ( E signo? dígitos )?      Ej: +3.14E-2   -120   42   +25
```

Estado	dígito	+	-	punto	E	$	Nota
0	2	1	1	—	—	—	**Estado inicial**
1	2	—	—	—	—	—	**Después del signo**
2	2	—	—	3	5	8	**Dígitos enteros**
3	4	—	—	—	—	—	**Después del punto**
4	4	—	—	—	5	8	**Dígitos decimales**
5	7	6	6	—	—	—	**Exponente E**
6	7	—	—	—	—	—	**Signo del exponente**
7	7	—	—	—	—	8	**Dígitos del exponente**
8	—	—	—	—	—	—	**ACEPTA**

Implementando un AFD con Tabla de Transiciones

Del diagrama al código

Algoritmo de reconocimiento

La tabla T[estado][símbolo] = estado_siguiente permite implementar cualquier AFD de forma genérica.

1. Inicializar estado = estado_inicial
2. Leer símbolo a símbolo
3. Consultar tabla para siguiente estado
4. Si estado_final → ACEPTAR, si no → RECHAZAR

```
estado = 0;
while (estado != 8 && estado != ERROR) {
  ch = siguiente_caracter();
  estado = T[estado][ch];
}
if (estado == 8)
  aceptar();
else
  error();
```

Ventaja de la tabla

El mismo código sirve para CUALQUIER AFD, solo cambia la tabla T. Esto es exactamente lo que hacen los generadores automáticos de analizadores léxicos como Lex/Flex.

Problema: tablas grandes

Si Σ tiene muchos símbolos, la tabla puede ser enorme. Solución: almacenar solo entradas no nulas y usar una tabla de índices por estado (compresión de tabla).

Ejemplo: para ASCII completo (256 símbolos) y 9 estados → tabla de 2304 entradas. Con compresión se puede reducir a solo las entradas no nulas.

04

SECCIÓN

Autómatas Finitos No Deterministas (AFN)

Múltiples caminos posibles

Autómatas Finitos No Deterministas (AFN)

La función de transición devuelve un conjunto de estados

Un AFN puede estar en varios estados simultáneamente. Se dice que 'conjetura' cuál es el camino correcto. La diferencia con el AFD está en la función de transición δ.

AFD vs AFN — Diferencia clave en la función de transición δ:

AFD

$\delta : Q \times \Sigma \rightarrow Q$

Dado un estado y un símbolo, devuelve EXACTAMENTE UN estado siguiente.

AFN

$\delta : Q \times \Sigma \rightarrow 2^Q$

Dado un estado y un símbolo, devuelve un SUBCONJUNTO de estados (puede ser vacío, uno o varios).

Definición formal AFN: $A = (Q, \Sigma, \delta, q_0, F)$

Q	Conjunto finito de estados.
Σ	Conjunto finito de símbolos de entrada.
$q_0 \in Q$	Estado inicial.
$F \subseteq Q$	Conjunto de estados de aceptación.

PREGUNTA PARA LA CLASE

¿Cuál es la principal diferencia entre un AFD y un AFN?

A) El AFN no tiene estados de aceptación

B) En el AFN δ devuelve un conjunto de estados

C) El AFD no puede leer cadenas vacías

D) El AFN solo acepta lenguajes regulares

Pista: La diferencia está en el tipo de función de transición δ.

05

SECCIÓN

Construcción de Thompson

De ER a AFN paso a paso

Construcción de Thompson

Algoritmo para convertir una ER en un AFN

¿Qué es la Construcción de Thompson?

Es un algoritmo que, dada una expresión regular r, construye un AFN que acepta exactamente L(r). Es la base de los motores de expresiones regulares modernos.

Paso 1: ER básicas

Se construye un AFN mínimo para cada caso base: carácter 'a', cadena vacía ε, lenguaje vacío ∅.

Paso 2: Concatenación rs

Se conecta el AFN de r con el de s mediante una ε-transición entre el estado final de r y el inicial de s.

Paso 3: Alternativa r|s

Se agrega un nuevo estado inicial con ε-transiciones a los iniciales de r y s, y los estados finales se unen con ε al nuevo final.

Paso 4: Cerradura r*

Se agregan ε-transiciones para permitir cero o más repeticiones: del final al inicial del subAFN, y del nuevo inicial al nuevo final.

Thompson: Casos Base (ER Básicas)

Los bloques de construcción del algoritmo

Toda expresión regular básica es de la forma a, ε o ∅

Carácter 'a'

```
i →(a)→ f*
```

Un AFN con dos estados:
→ estado inicial i
→ estado final f (aceptación)
Conectados con una transición etiquetada 'a'.

L(AFN) = {a}

Cadena vacía ε

```
i →(ε)→ f*
```

Un AFN con dos estados:
→ estado inicial i
→ estado final f (aceptación)
Conectados con una ε-transición.

L(AFN) = {ε}

Lenguaje vacío ∅

```
i    f*   (sin arco)
```

Un AFN con dos estados:
→ estado inicial i
→ estado final f (aceptación)
Sin ninguna transición entre ellos.

L(AFN) = {} (vacío)

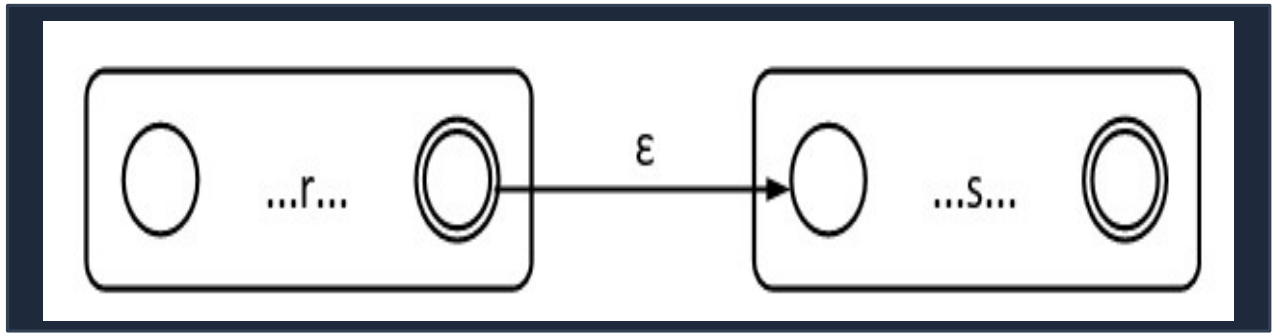
Thompson: Concatenación y Alternativa

Componiendo AFNs a partir de subAFNs

Concatenación: AFN para rs

Idea

Se conecta el estado final del AFN de r con el estado inicial del AFN de s mediante una ε-transición. El nuevo estado inicial es el de r y el nuevo estado final es el de s.

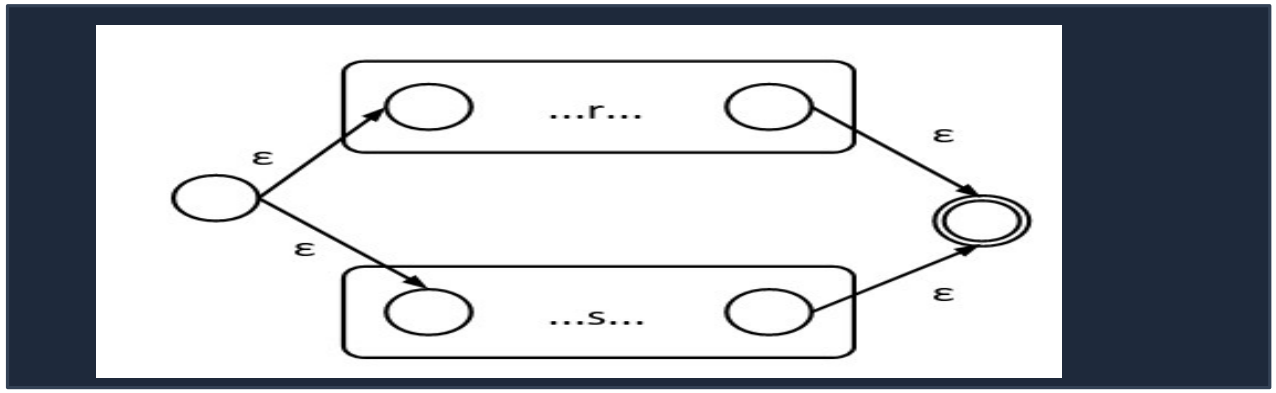


Alternativa: AFN para r | s

Idea

Se crea un nuevo estado inicial i con ε-transiciones a i_r e i_s.
Se crea un nuevo estado final f, con ε-transiciones desde f_r y f_s.

Así el autómata puede 'elegir' por cuál camino ir.

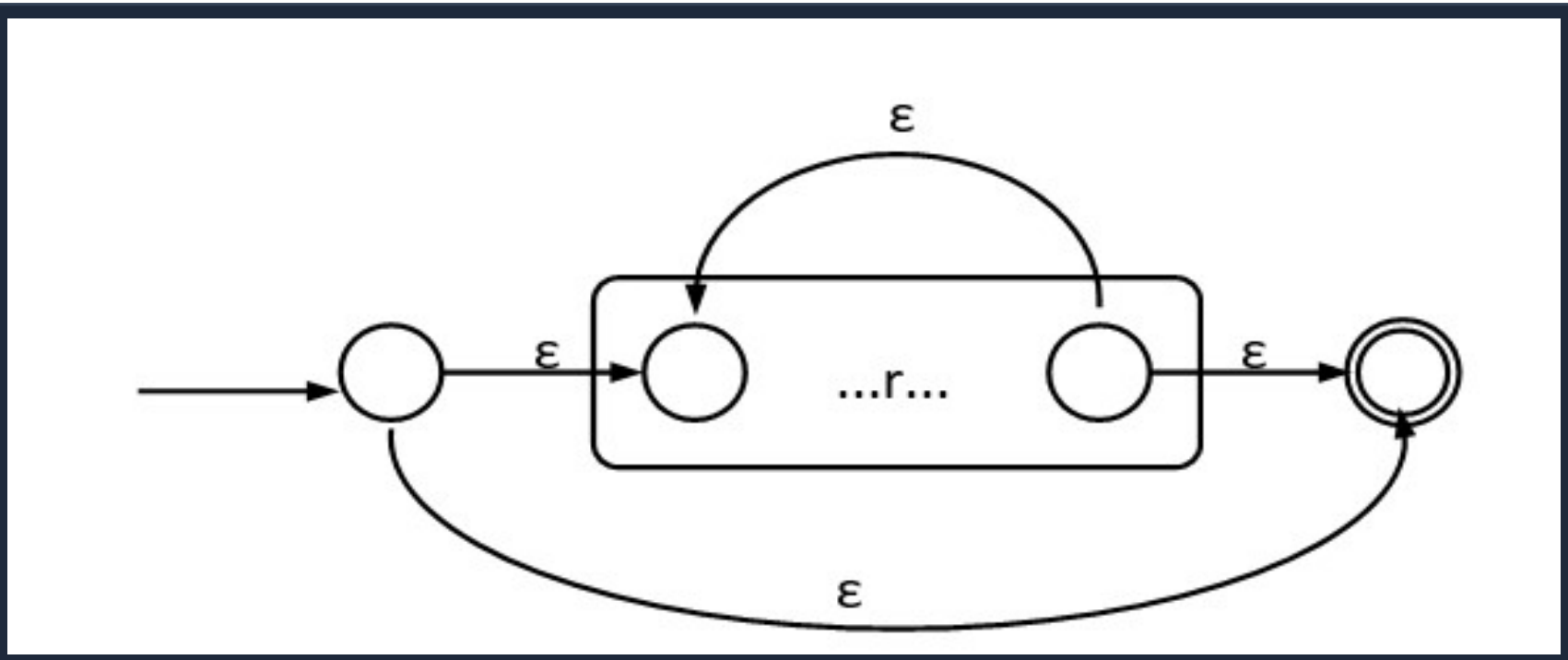


Thompson: Cerradura de Kleene r*

El caso más elaborado del algoritmo

Idea de la construcción

Para construir el AFN de r* se necesita permitir: (1) cero repeticiones de r (aceptar ε), y (2) una o más repeticiones. Se logra con cuatro ε-transiciones estratégicas: dos del nuevo estado inicial y dos del estado final de r.



Ejemplo Completo: Thompson para (a|b)*abb

Construyendo el AFN paso a paso

Paso 1: AFN para 'a' y 'b' (casos base)

```
Se crean dos AFNs mínimos:
  AFN_a:  q0 →(a)→ q1*
  AFN_b:  q2 →(b)→ q3*
Cada uno tiene un estado inicial y uno de
aceptación.
```

Paso 3: AFN para (a|b)* (cerradura)

```
Se agregan 4 ε-transiciones al AFN de a|b:
  nuevo_i →ε→ q_i    (entrar)
  nuevo_i →ε→ nuevo_f    (cero rep.)
  q_f →ε→ q_i    (repetir)
  q_f →ε→ nuevo_f    (salir)
```

Paso 2: AFN para a | b (alternativa)

```
Nuevo estado inicial q_i con ε→q0 y ε→q2.
Nuevo estado final q_f con q1→ε y q3→ε hacia q_f.
  q_i →ε→ q0 →(a)→ q1 →ε→ q_f*
  q_i →ε→ q2 →(b)→ q3 →ε→ q_f*
```

Paso 4: AFN para (a|b)* abb (concatenación)

```
Se encadena el AFN de (a|b)* con tres AFNs base:
  AFN(a|b)* → ε → AFN_a → ε → AFN_b → ε → AFN_b
El estado final del último AFN_b es el estado de
aceptación.
```

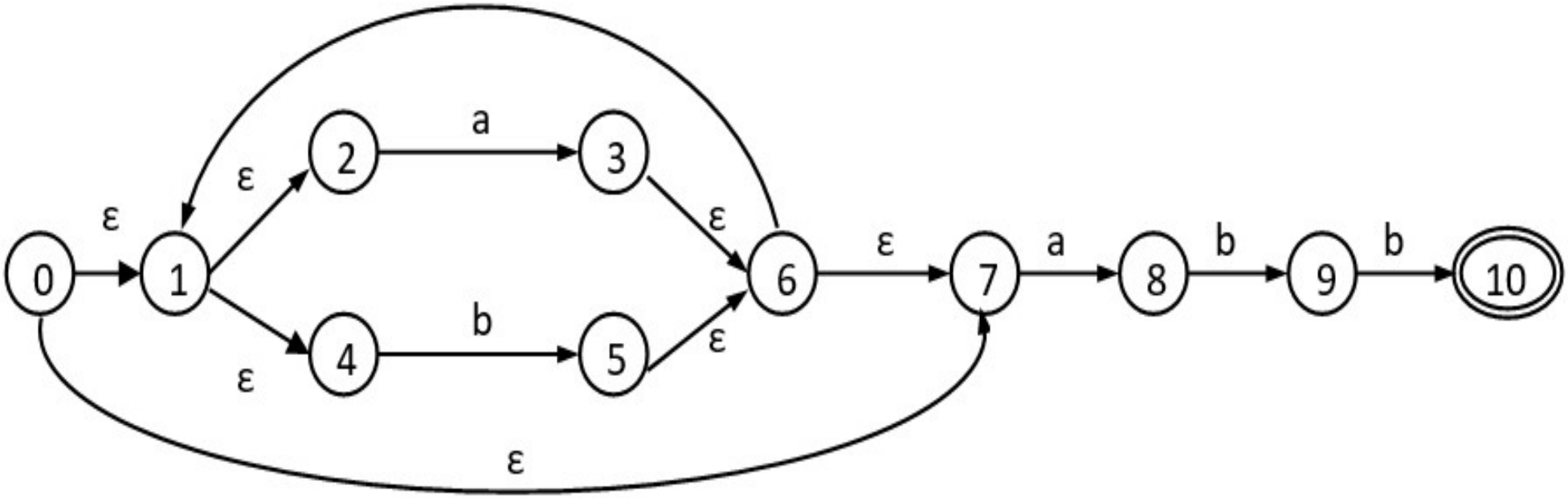
El AFN resultante acepta exactamente las cadenas sobre {a,b} que terminan en 'abb'

Ejemplo AFN: reconocer cadenas que terminan en 'abb'

ER: (a | b) abb*

La ER (a|b)*abb reconoce todas las cadenas sobre {a,b} que terminan en 'abb'.
Ej. aceptadas: abb, aabb, babb, aaabb Ej. rechazadas: ab, ba, abba, bb

Diagrama del AFN:



PREGUNTA PARA LA CLASE

En la Construcción de Thompson para r*, ¿cuántas ε-transiciones se agregan?

A) Una (del final al inicio)

B) Dos (ida y vuelta)

C) Cuatro (para cubrir todos los casos)

D) Ninguna, no usa ε-transiciones

Pista: Se necesita cubrir: entrar, cero repeticiones, repetir y salir.

Ejemplo AFN: reconocer cadenas

La ER ((c|a)b*)*

Diagrama del AFN:

06

SECCIÓN

Síntesis y Cierre

Consolidando lo aprendido

Resumen de la Clase

Conceptos clave en una sola vista

Autómata Finito

Modelo matemático con estados, transiciones, estado inicial y estados de aceptación. Reconoce lenguajes regulares.

AFN

La función $\delta: Q \times \Sigma \rightarrow 2^Q$ devuelve un subconjunto de estados. Puede estar en varios estados simultáneamente.

AFD — Definición formal

$M = (Q, \Sigma, \delta, q_0, F)$. La función $\delta: Q \times \Sigma \rightarrow Q$ devuelve exactamente un estado siguiente.

AFD ≡ AFN (equivalencia)

Todo AFN puede convertirse a un AFD equivalente (construcción de subconjuntos). Igual potencia expresiva.

Tabla de transiciones

Representación tabular del AFD. Permite implementar cualquier AFD con el mismo código genérico.

Construcción de Thompson

Algoritmo para convertir ER → AFN. Opera recursivamente sobre la estructura de la ER usando ε-transiciones.

Mapa del Camino: ER → AFN → AFD

Las tres representaciones equivalentes

Expresión Regular (ER) — Thompson → **Autómata Finito No Determinista (AFN)** — Subconjuntos → **Autómata Finito Determinista (AFD)**

ER

Notación declarativa compacta. Fácil de leer y escribir. Necesita ser convertida para ejecutarse.

AFN

Fácil de construir desde ER (Thompson). No determinista: puede estar en múltiples estados.

AFD

Eficiente para ejecutar: siempre un estado activo. Se usa en la implementación real.

Teorema de Kleene: ER, AFN y AFD reconocen exactamente la misma clase de lenguajes: los lenguajes regulares.

¿Preguntas?

Autómatas y Gramáticas | Ingeniería en Informática | 3° Año

Visualizador de autómatas online: https://automata.cs.ru.nl

Simulador de ER interactivo: https://regexr.com